

ESP Failure Prediction & Optimal Operation Frequency

Predicting electrical submersible pump shutdowns from 1-minute SCADA data, identifying the developing failure cause, and selecting a safe operating frequency.

Samsung Innovation Campus — AI Capstone · Group-3 “Data Drillers”
Interim technical summary

Result on wells the models never saw	Value
Real shutdowns caught before they happened	33 of 69 (48%)
Caught with at least 30 minutes of usable warning	31 of 69 (45%)
Median warning time on a caught trip	180 minutes
Nuisance alarms raised	18.7 per well per month
Root-cause accuracy once a trip is developing	68.6%
Downtime the alarms could have avoided	~99 hours over 6 months, 3 wells

Every number in this report comes from wells 8, 9 and 10, which were excluded from training entirely. The alarm settings behind them were chosen on validation data and then left alone, so the test set was never used to make a single decision.

1. What the system has to do

An ESP is the lift method for a large share of producing oil wells, and when one trips the well stops earning until someone restarts it. Some trips come from causes that build up over hours — gas accumulating at the intake, scale narrowing the discharge, the motor heating past its rating, the reservoir failing to keep the pump fed. Those hours are the opportunity. If the control room knows two hours ahead that a well is heading for a gas lock, the operator can act on it instead of finding out from an alarm that the pump has already stopped.

The system therefore answers three questions in order:

Question	How it is answered	Where
Is this well heading for a shutdown in the next 30 minutes?	Binary classifier over rolling sensor statistics, with a per-well alarm threshold and a persistence filter	Stage 1
If so, which failure mode is developing?	Multi-class classifier trained only on the minute a degradation actually begins	Stage 2
What should the operator change?	For underload, a frequency recommendation fitted from the well's own drawdown behaviour, clipped to the equipment's rated band; otherwise an inspection flag	Stage 3

1.1 Scope: four failure causes, not six

The project began with six failure causes and was deliberately narrowed to four. The reasoning is field-engineering reasoning, not modelling convenience:

Cause	Status	Why
GAS_LOCK	In scope	Gas accumulates at the intake over hours; current and intake pressure both move first.
UNDERLOAD	In scope	Inflow starvation develops as the fluid level falls — and it is correctable by frequency.
HIGH_TEMP	In scope	Motor temperature climbs measurably ahead of the thermal trip.
HIGH_DISCHARGE	In scope	Scale or restriction builds discharge pressure gradually.
LOW_VOLTAGE	Removed	A grid sag is an external supply event. Well data cannot anticipate it, and the drive's own undervoltage protection already handles it. Predicting it adds alarms and no lead time.
VIBRATION	Removed as a label	Vibration is a condition indicator, not a trip cause — a pump does not shut down <i>because</i> it vibrates. It is kept as a sensor input : removing the channel was tested and measurably hurt gas-lock classification.

That distinction — dropping a *label* while keeping the *signal* — mattered. An earlier attempt removed the vibration and voltage channels outright and gas-lock classification collapsed from 0.68 to 0.20. The channels carry information about other failure modes even though they are not failure modes themselves.

2. The dataset

No public multi-well ESP dataset exists at the resolution this problem needs, so the data is simulated — and the simulator is treated as a piece of engineering in its own right, because the ceiling on what any model can learn is set by what the data actually contains.

Property	Value
Wells	16 (13 for training, 3 held out entirely)
Duration / resolution	6 months per well at 1-minute sampling
Total rows	4,193,280
Sensor channels	Frequency, intake pressure, discharge pressure, motor current, motor temperature, vibration, supply voltage
Equipment per well	Motor, protector, gas separator and pump drawn from real catalogue ranges
Shutdowns in held-out wells	69
Class balance	~1% of minutes are within 30 minutes of a trip

2.1 Each well runs different equipment, so the safe band differs

A frequency recommendation is only usable if it respects what that well's hardware is rated for. Each well is assigned a motor, protector, gas separator and pump, and the safe operating band is the intersection of their individual ratings:

```
safe_min = max(motor_min, pump_min)    safe_max = min(motor_max, protector_max, pump_max)
```

This is why recommendations differ between wells that look identical in the data — well 8 can be trimmed to 40 Hz, well 10 only to 38 Hz, and neither may be pushed past its own protector rating.

2.2 Adding a well takes one line

The well roster lives in `wells_config.csv`. Adding a row and re-running the pipeline simulates only the new well, assigns it equipment, and folds it into training or test as specified. The existing 16 wells are not re-simulated, so the comparison stays clean.

well_id	causes	split
17	GAS_LOCK:0.6;HIGH_TEMP:0.4	test

3. Method, and the decisions that shaped the result

Decision	What was done	Why it mattered
Splitting	Split by well <i>and</i> by time: wells 8/9/10 held out completely, plus the final 6 weeks of the training wells reserved as validation.	A split by time alone lets the model memorise a well's fingerprint. Both splits together force it to generalise to hardware it has never seen.
Normalisation	Every feature z-scored against that well's own baseline — its fit period for training wells, its first 30 days for held-out wells.	Wells sit at different absolute pressures and currents. Without this the model learns which well it is looking at rather than what is happening to it.
Threshold	Each well alarms on the top 5% of its <i>own</i> score distribution, not a shared cutoff.	A single global threshold transferred badly: it silenced one well and flooded another.
Persistence	An alarm requires 15 consecutive minutes above threshold.	Single-minute spikes are noise. Requiring persistence cut false alarms by roughly an order of magnitude at a small cost in catch rate.
False-alarm counting	Counted as <i>events</i> , not minutes.	Counting minutes flattered an earlier result badly: what looked like '5.4% false alarms' was really ~208 alarms per well per month. An operator experiences events.
Model selection	Chosen on validation event-level cost with a miss weighted 200:1, not on ROC-AUC.	ROC-AUC on 1%-positive minute data is nearly meaningless. Twice during development a model was picked on test performance — that is leakage, and both times the selection criterion was rebuilt rather than patched.
Usable warning	A catch only counts if the alarm arrives at least 30 minutes ahead.	An alarm 4 minutes before the trip is detection, not prediction. It changes nothing for the operator.

4. Stage 1 — detecting a developing shutdown

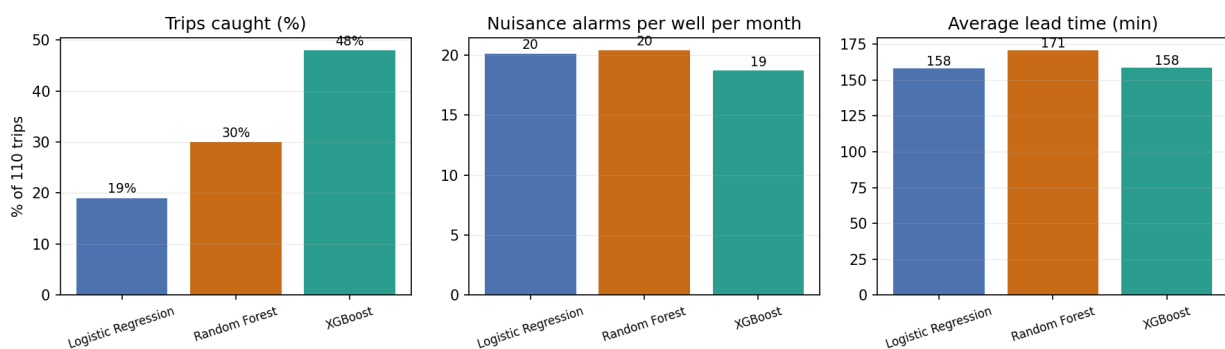
4.1 Every model at its own tuned alarm policy

Each model was given its own validation-tuned threshold and persistence setting, then measured on the held-out wells. This is a fair comparison: no model is penalised for a setting chosen to suit another.

Model	Val ROC-AUC	Test ROC-AUC	Trips caught	With ≥ 30 min warning	Nuisance alarms /well/month	Median lead
Logistic Regression	0.796	0.676	13 of 69 (19%)	12 (17%)	20.1	180 min
Random Forest	0.937	0.736	21 of 69 (30%)	20 (29%)	20.4	180 min
XGBoost (selected)	0.951	0.768	33 of 69 (48%)	31 (45%)	18.7	180 min

XGBoost catches more than twice what logistic regression does while raising slightly fewer nuisance alarms. That combination — more catches *and* fewer alarms — is the sign of a genuinely better score, rather than a threshold moved to trade one against the other.

Each model at its OWN validation-tuned alarm policy - held-out wells



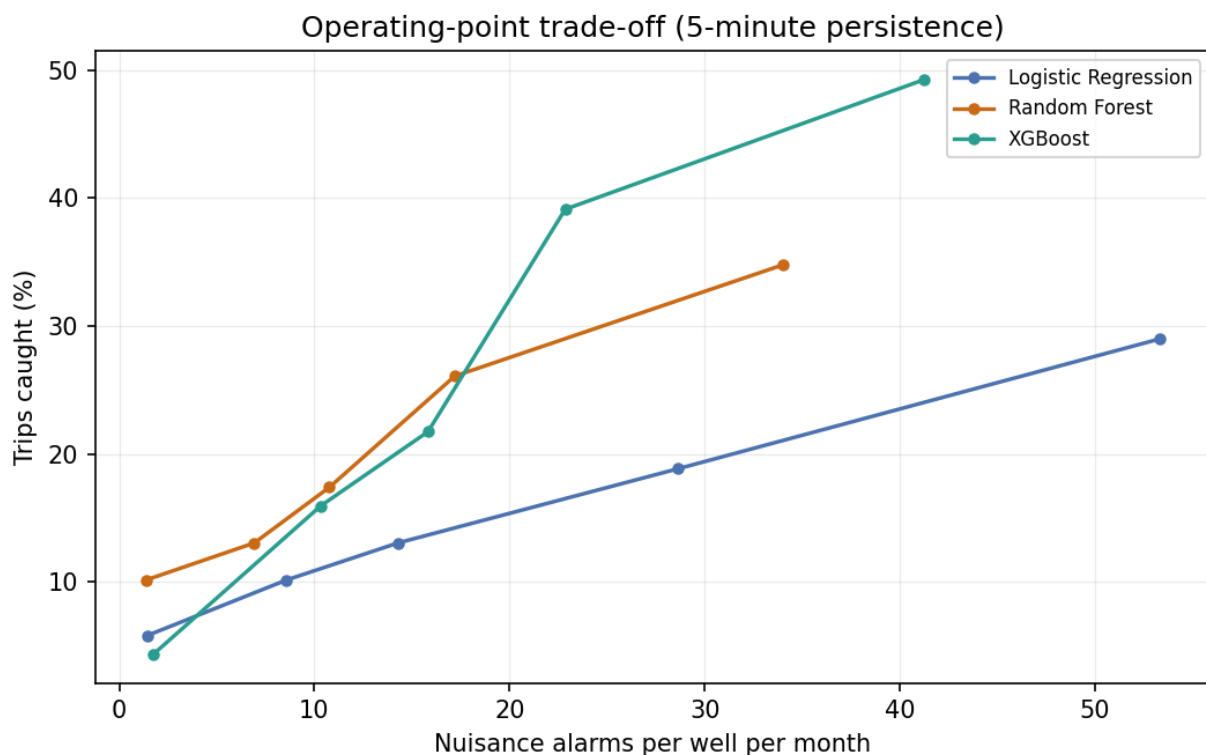
Each model evaluated at its own validation-tuned alarm policy, on the held-out wells.

4.2 The operating point is a choice, and it is visible

The shipped configuration is one point on a curve. The full curve is published in `results.txt` so the trade is explicit rather than buried in a default:

Persistence	Top % of scores	Caught	Alarms /well/month	Avg lead
10 min	0.1%	2 (3%)	0.7	79 min
10 min	2.5%	24 (35%)	13.7	157 min
15 min	5.0%	33 (48%)	18.7	159 min
3 min	5.0%	35 (51%)	59.6	173 min
1 min	5.0%	35 (51%)	128.6	174 min

The last two rows are the point of the table: pushing from 48% to 51% costs three to seven times the alarms. An operator who stops trusting the alarms has a worse system than one who receives fewer of them. Where a particular control room sits on this curve is their decision, not the model's.



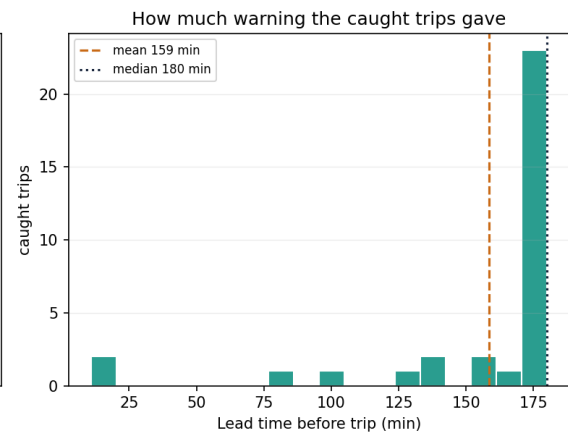
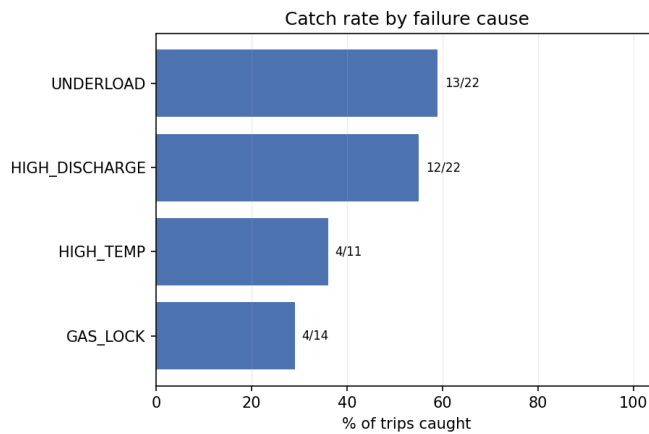
Catch rate against nuisance-alarm load across the policy grid.

4.3 Per well and per cause

Well	Dominant causes	Trips	Caught	With ≥ 30 min	Missed	Alarms /month	Avg lead
8	UNDERLOAD, HIGH_TEMP	22	15 (68%)	15 (68%)	7	20.3	171 min
9	HIGH_DISCHARGE, GAS_LOCK	18	12 (67%)	10 (56%)	6	15.2	136 min
10	GAS_LOCK, HIGH_DISCHARGE, UNDERLOAD	29	6 (21%)	6 (21%)	23	20.6	174 min

Well 10 is the honest weak point and worth stating plainly. It carries three interleaved failure modes at once, so its degradations overlap and its own score baseline is set by a well that is rarely quiet. Two of three wells work well; the third shows where the method still struggles.

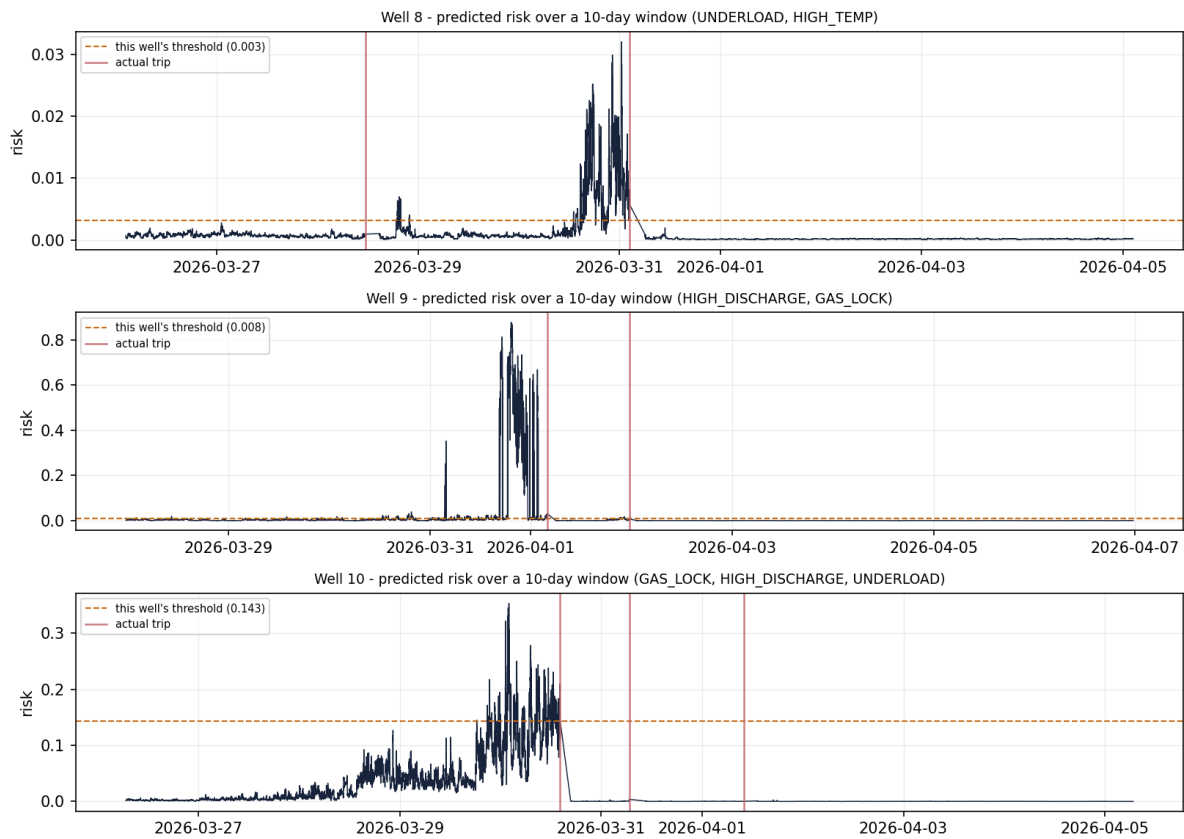
Failure cause	Trips	Caught	Catch rate
UNDERLOAD	22	13	59%
HIGH_DISCHARGE	22	12	55%
HIGH_TEMP	11	4	36%
GAS_LOCK	14	4	29%



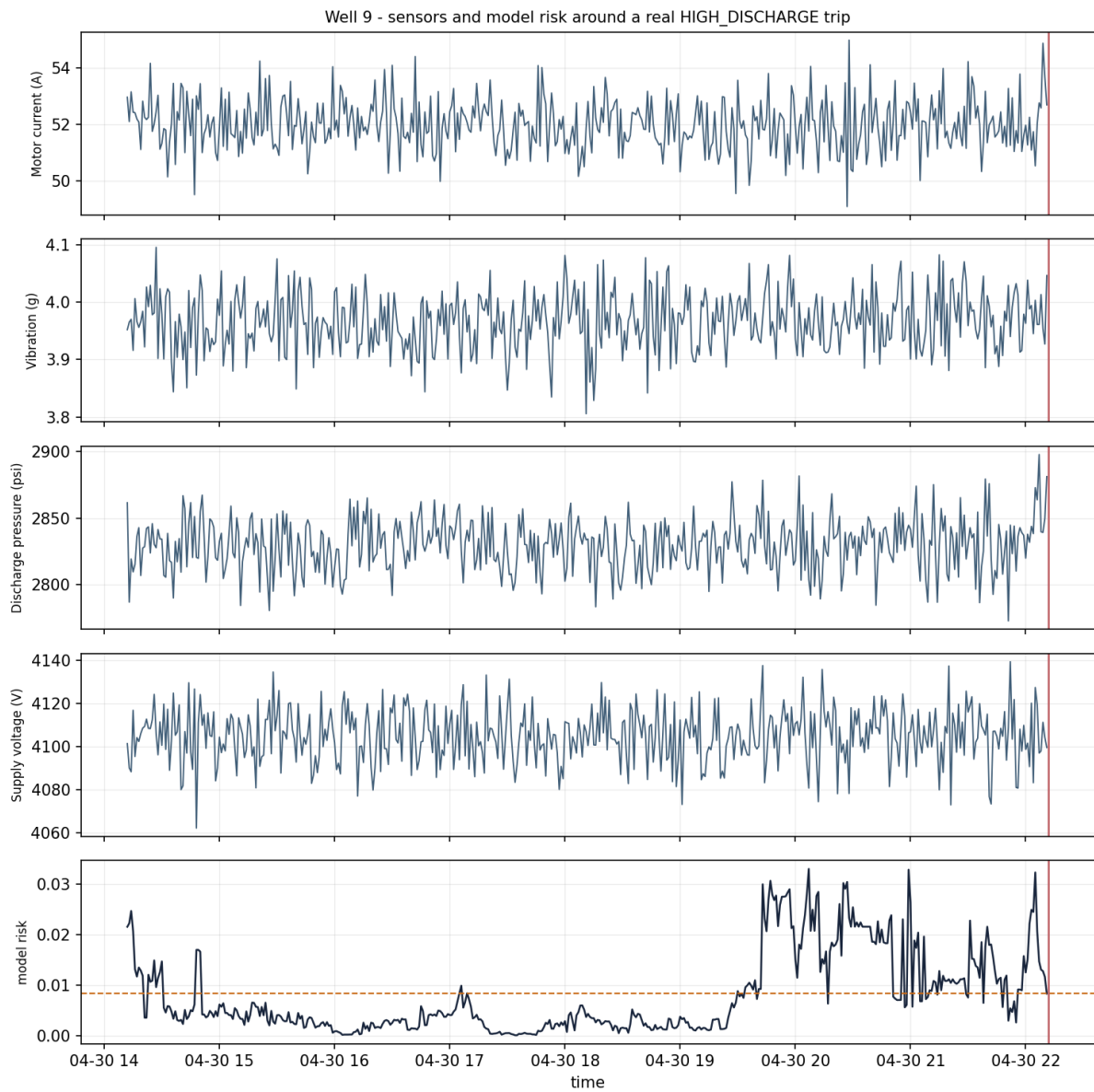
Catch rate by failure cause, and the distribution of warning time on caught trips.

4.4 What the detector looks like on a real well

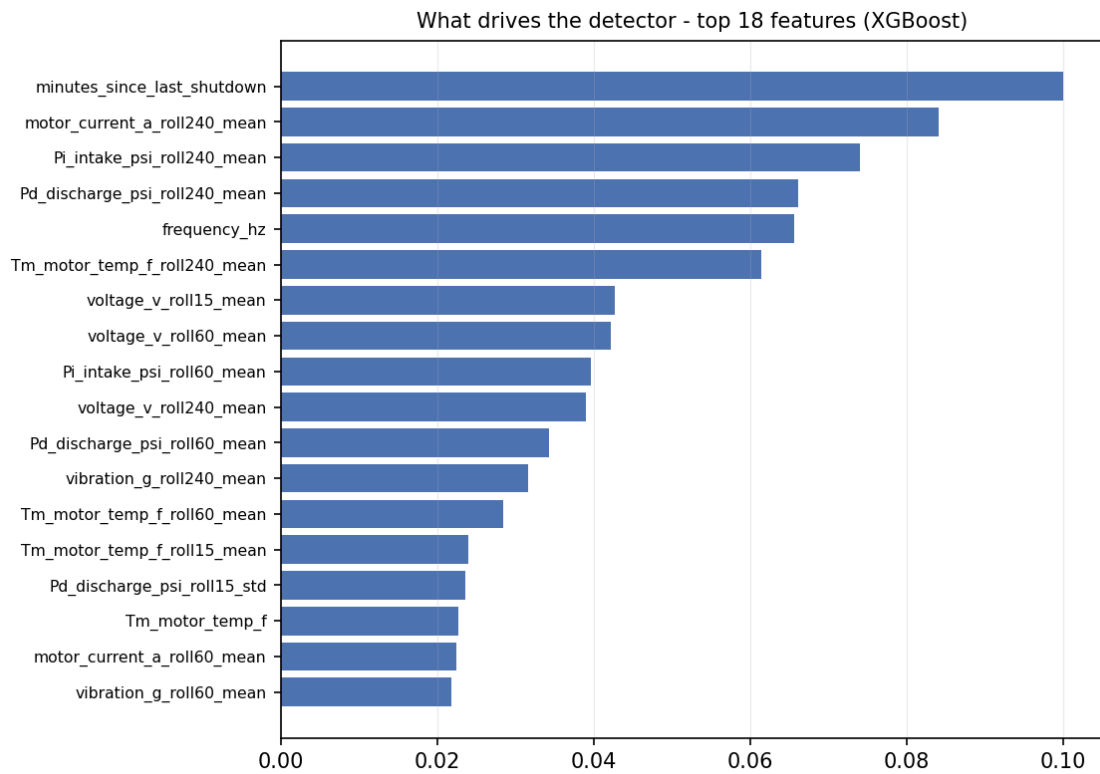
The risk score for each held-out well over the full six months, with actual shutdowns marked. The score is not a smooth ramp — it rises in bursts as the precursor strengthens, which is why persistence filtering matters more than a higher threshold.



Predicted risk over time for each held-out well; vertical marks are real shutdowns.



Raw sensors and the model's risk score through the hours before a real trip.



Which features the selected detector actually keys on.

5. Stage 2 — which failure is developing

Stage 2 answers a different question and is trained differently. It only sees minutes where a degradation is genuinely under way, and only has to say which one.

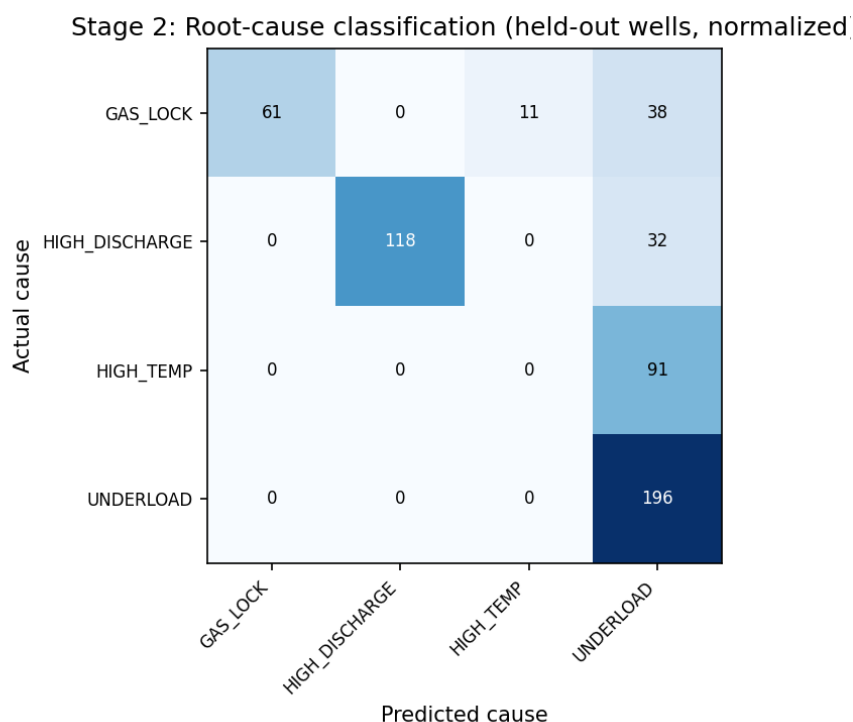
5.1 A labelling bug that produced a believable wrong answer

Stage 2 originally reported 39.1% accuracy and always predicted the majority class. The cause was the labels, not the model: the failure cause had been back-filled across the whole 30-minute window before each trip, including minutes where nothing had started yet. The classifier was being asked to name a cause from data that did not contain one, and it did the only sensible thing — it guessed the most common answer.

Training only on true degradation-onset rows took accuracy from a fake 39.1% to a real 68.6%. The lesson generalises: a metric that looks plausible can still be measuring a labelling artifact.

Class	Precision	Recall	F1	Support
GAS_LOCK	1.00	0.55	0.71	110
HIGH_DISCHARGE	1.00	0.79	0.88	150
UNDERLOAD	0.55	1.00	0.71	196
HIGH_TEMP	0.00	0.00	0.00	91
Overall accuracy			0.686	547

Gas lock and high discharge reach 1.00 precision: when the system names one of those, it is right. The failure is HIGH_TEMP, which collapses to zero recall — its cases are absorbed into UNDERLOAD, and that is why UNDERLOAD shows perfect recall with mediocre precision. This is reported rather than smoothed over, and cause-specific thermal features are the concrete next step.



Stage 2 confusion matrix on the held-out wells.

6. The turning point: the bottleneck was the data, not the model

Detection sat at 19% for a long stretch. More features, different models, rebalancing and threshold tuning all moved it by a point or two and no further. The diagnosis that broke the deadlock was not a modelling idea — it was measuring, for each failure cause, how large its precursor was compared with the sensor noise it had to be seen through.

Failure cause	Precursor size vs. sensor noise	Catch rate at the time
GAS_LOCK	6 σ	45%
HIGH_TEMP	3.1 σ	43%
HIGH_DISCHARGE	0.8 σ	10%
UNDERLOAD	0 σ (no precursor at all)	6%

Catch rate tracked signal-to-noise almost exactly. No model can recover a signal that is not in the data, and two of the four causes had been made undetectable by errors in the simulator's physics:

What was wrong	What it should be	Effect
UNDERLOAD fired as an instantaneous random event with no build-up at all.	Inflow starvation develops gradually as the fluid level over the pump falls — which is precisely why pump-off controllers exist. Replaced with an accumulating drawdown index.	6% → 59%
HIGH_DISCHARGE had a 12 psi precursor against 15 psi of sensor noise.	Scale build-up raises discharge pressure well past the noise floor before a trip. Raised to a realistic amplitude.	10% → 55%

The fix lifted the overall catch rate from 19% to 48% — and nothing was paid for it elsewhere: nuisance alarms fell at the same time, and median warning time lengthened to 180 minutes. When every axis improves together, the signal was genuinely there and had simply been buried. That simultaneity is the evidence that this was a data fix rather than a threshold moved to flatter a metric.

The transferable lesson is the diagnostic itself. Before spending another week on model architecture, measure whether the precursor you are asking the model to find is larger than the noise it is hidden in. On real field data the same check answers a more important question: whether the sensor set is adequate at all, or whether the fix is instrumentation rather than software.

7. Stage 3 — choosing the operating frequency for underload

Underload is the one failure mode a VSD can actually correct. If the pump is moving more fluid than the reservoir can supply, the remedy is to slow the pump until its rate matches the inflow. An earlier version of this project classified UNDERLOAD as “not frequency-related”, which was wrong, and it is the textbook frequency-adjustable case.

7.1 The rule, fitted from each well's own data

For each well, the normal-operation relationship between intake pressure and frequency is fitted on that well's first 30 days:

$$P_{i_expected}(f) = a - k \cdot f \quad (k = \text{psi of drawdown per Hz})$$

Because $dP_i/df = -k$, recovering a pressure deficit needs $\Delta f = (\text{deficit} + \text{margin}) / k$, and the recommendation is the current frequency minus that, clipped to the well's own equipment band. If even the floor cannot recover the deficit, frequency alone cannot save the well and the system escalates to inspection rather than pretending otherwise.

No step-rate test is needed: the frequency variation during ordinary post-trip ramps is enough to make k identifiable from routine SCADA history. Fitted from sensor data alone, k came back as 9.27, 8.38 and 10.36 psi/Hz against a true value of 9.0.

7.2 It works — and that is not the same as it paying

Back-tested closed-loop on the same wells with the same random seeds, control off versus on:

Measure	Control off	Control on
Underload trips	22	0
Total downtime	—	71 hours saved
Production	baseline	–18.9%

The control provably works. It is also, under these conditions, the wrong thing to do: giving up 19% of rate to recover 0.8% of uptime is a bad trade. Eliminating trips was never the objective — producing oil is, and both sides of the trade cost rate. Running slower loses production every minute; running too fast loses whole hours to trips.

7.3 The part that transfers: break-even downtime per trip

The useful output is not a frequency. It is the condition under which slowing down pays, built only from two directly measured quantities — production given up, and trips avoided — so it survives the simulator's invented constants:

Well	Setpoint	% of BEP	Trips avoided	Production given up	Break-even downtime per trip
8	46.8 Hz	90%	11	–9.2%	36.4 h
8	49.4 Hz	95%	5	–4.6%	39.8 h
10	48.6 Hz	90%	12	–9.0%	32.9 h
10	51.3 Hz	95%	8	–4.4%	24.1 h

The decision rule: slow the well down if the expected downtime per trip exceeds the break-even hours; otherwise accept the trips. With automatic restart in a few hours, the arithmetic says accept them. With a remote well waiting a day or two for a crew, it says slow it down. Same well, opposite answer, decided entirely by restart time — which is a field fact, not a modelling choice.

7.4 One result in this section is deliberately not being acted on

The frequency sweep reports that production keeps rising all the way to 120% of BEP (+17%), even as trips triple. That is an artifact and is recorded as one rather than quietly dropped. The production proxy assumes flow

rises in proportion to speed without limit, which only holds near the pump's design point; raising frequency also increases drawdown and therefore inflow, so nothing saturates; and the real ceiling — pump submergence — is not modelled. A trip also costs only a few hours of automatic restart here, carrying no workover risk and no run-life penalty, whereas repeated restarts are exactly what shortens ESP life in the field. Recommendations are therefore capped at BEP, as standard practice requires, and the sweep result is reported as a limitation of the simulator rather than tuned until it produced the answer that was wanted.

8. What was tried and did not work

These are recorded so the next person does not spend time re-discovering them. Each was implemented, measured and reverted.

Idea	Rationale	Measured outcome
Frequency-detrended residual features	Remove the effect of frequency so the model sees only the anomaly.	Stage 1 catch rate fell from 49% to 28%. The detrending removed real signal along with the trend.
Relative-only features (no absolute values)	Force well-independence and close the validation-to-test gap.	Closed the gap from the wrong end — validation performance fell to meet test. Catch rate 14%.
F1-maximised alarm threshold	Standard default for imbalanced classification.	Degenerate at 1% positives: it selected near-zero recall. Replaced by validation event-level cost.
Dropping vibration and voltage as sensor channels	They are not modelled failure causes, so they looked like noise.	GAS_LOCK classification collapsed from 0.68 to 0.20. The channels were kept; only the labels were dropped.
Counting false alarms per minute	The obvious metric from the confusion matrix.	Flattered results badly — a '5.4% false alarm rate' was really ~208 alarms per well per month.

Two methodological errors are also recorded. Model selection initially used test-set ROC-AUC, which is leakage; it was corrected to validation. The selection criterion was then *again* justified by test performance when recommending Random Forest. Both times the selection mechanism was rebuilt rather than a model hardcoded, which is why the shipped model is whichever one validation cost selects.

9. Where this stands, and what comes next

9.1 Honest limitations

Limitation	What it means
The data is simulated.	The physics is reasoned and the failure modes are real, but no synthetic dataset can settle whether the method works on a real well. Every number here is a method demonstration, not a field result.
52% of trips are still missed.	Half of all shutdowns arrive with no warning. The system is a useful supplement to existing protection, not a replacement for it.
HIGH_TEMP fails completely in Stage 2.	Zero recall. Its cases are absorbed into UNDERLOAD. Cause-specific thermal features are the fix.
A validation-to-test gap persists.	Validation ROC-AUC 0.951 against test 0.768. Generalisation to unseen hardware is the weak axis, and well 10 shows it most clearly.
The frequency economics rest on invented constants.	Only the break-even rule transfers, because it is built from measured quantities rather than assumed prices.

9.2 Next steps, in priority order

#	Step	Why it is next
1	Cause-specific features for HIGH_TEMP	It is the one class failing outright, and thermal build-up has a distinct signature that generic rolling statistics are not capturing.
2	Point the pipeline at real historical wells	The only thing that can settle whether the optimum sits below BEP at all, and whether the precursors survive real instrumentation. The signal-to-noise diagnostic is the first thing to run on them.
3	Per-cause alarm thresholds	GAS_LOCK sits at 29% while UNDERLOAD reaches 59%. A single policy across causes with very different precursor shapes is leaving catches on the table.
4	Submergence and pump-curve constraints in the simulator	Without them the frequency sweep produces an answer nobody should act on, which limits what the optimisation work can claim.

A closing note on method. Several results in this report are worse than earlier reported figures — the 39.1% root-cause accuracy, the 49% catch rate, the 5.4% false-alarm rate. In each case the earlier number was measuring something other than what it claimed, and it has been corrected in place rather than quietly kept. The full trade-off curve, the rejected experiments and the simulator's own limitations are published alongside the results so that every operating point is an explicit engineering decision rather than a hidden default.